

Lecture 06 : IP addressing and routing (Part I)

Introduction

In the vast landscape of the internet, the seamless flow of information relies heavily on fundamental protocols, with TCP/IP standing as a cornerstone. At the heart of this architecture lies the IP (Internet Protocol) layer, operating at the network level, which is primarily responsible for the intricate tasks of packet routing and addressing. This lecture delves into the critical mechanisms that enable IP to manage data transmission across diverse networks, specifically focusing on IP packet fragmentation. Understanding how IP handles packets that are too large for certain network segments is crucial for comprehending the internet's robustness and efficiency.

The necessity for fragmentation arises because different networks have varying capacities for the maximum size of data units they can handle. When a large packet encounters a network with a smaller capacity, it must be broken down into smaller, manageable pieces. This process, known as fragmentation, ensures that data can traverse heterogeneous network environments. We will explore two primary approaches to fragmentation—transparent and non-transparent—and examine the specific fields within the IP header that facilitate these operations, ultimately revealing how IP ensures data delivery even under challenging network conditions.

Core Concepts

Term	Definition	Significance
IP Fragmentation	The process of breaking down a large IP packet into smaller packets (fragments) when it encounters a network segment with a Maximum Transfer Unit (MTU) smaller than the packet's size.	Essential for enabling large data packets to traverse heterogeneous networks with varying size limitations, ensuring data can reach its destination.
Reassembly	The process of putting fragmented packets back together in their original order	Crucial for restoring the integrity of the original data unit after fragmentation, allowing the destination

Term	Definition	Significance
	to reconstruct the complete, original IP packet.	host to process the complete message.
Maximum Transfer Unit (MTU)	The largest size of a data unit (in bytes) that a network layer can handle or transmit without fragmentation.	Determines the maximum packet size a network can support, directly influencing when and where fragmentation might occur.
Transparent Fragmentation	A fragmentation method where fragmentation and reassembly occur within the same network, making the process invisible to subsequent networks.	Simplifies handling for downstream networks as they receive the original, reassembled packet, but can lead to bottlenecks at exit routers.
Non-Transparent Fragmentation	A fragmentation method where fragments are not reassembled by intermediate routers but are forwarded independently to the final destination host for reassembly.	The method used by the IP protocol, offering better network resource utilization and throughput by allowing parallel fragment routing, but increasing header overhead.
Identification (ID)	A 16-bit field in the IP header that uniquely identifies the original datagram (packet) to which a fragment belongs.	Allows the destination host to group all fragments originating from the same original packet for correct reassembly.
Fragment Offset	A 13-bit field in the IP header that specifies the position of the current fragment's data relative to the beginning of the original unfragmented IP datagram, measured in multiples of 8 bytes.	Enables the destination host to correctly order and reconstruct the original packet from its fragments.
Flags (D and M)	A 3-bit field in the IP header, where two bits are defined:	The 'D' flag prevents fragmentation, while the 'M'

Term	Definition	Significance
	'D' (Do Not Fragment) and 'M' (More Fragments).	flag indicates whether more fragments are expected after the current one, aiding reassembly.
Datagram Service	A connectionless, unreliable service model where each packet (datagram) is treated independently, with no guarantees of delivery, order, or duplicate prevention.	Characterizes the basic nature of the IP layer, highlighting its focus on best-effort delivery rather than reliability, which is handled by higher layers.

Detailed Analysis

1. The Fundamentals of IP Packet Fragmentation

IP packet fragmentation is a necessary mechanism in internetworking due to the inherent differences in Maximum Transfer Unit (MTU) sizes across various networks. When a large IP packet, originating from a source host or network, attempts to traverse a network segment with a smaller MTU, the packet must be broken down into smaller, compliant fragments. This process ensures that data can flow through diverse network infrastructures without being discarded due to size constraints.

The IP layer, residing at the network level, is responsible for this fragmentation. When a packet is handed to the IP layer for routing, if its size exceeds the MTU of the next hop network, the IP software or router will fragment it. Each resulting fragment is then treated as an independent IP packet for transmission. Conversely, at some point, these smaller packets must be reassembled to reconstruct the original data unit, a process known as reassembly. This reassembly typically occurs either within an intermediate network (in transparent fragmentation) or at the final destination host (in non-transparent fragmentation).

It's important to remember that IP operates as a basic datagram service. This means it is connectionless and inherently unreliable. IP does not guarantee packet delivery, order, or prevent duplicates. Each fragment, once created, is sent out as an independent entity, and the responsibility for reliability often falls to higher-layer protocols like TCP.

2. Transparent Fragmentation: Network-Internal Management

Transparent fragmentation is a method where the entire fragmentation and reassembly process occurs within a single network, making it "transparent" to subsequent networks. When a large packet enters a network whose MTU is smaller than the packet size, an ingress router fragments the packet. These fragments are then routed within that network. A key constraint of transparent fragmentation is that all fragments belonging to the same original packet must be directed to the same exit router within that network.

This designated exit router (let's call it RE) is responsible for reassembling all the fragments back into the original packet. Once reassembled, the original packet is then forwarded to the next network. From the perspective of the next network, it receives a complete, unfragmented packet, unaware that any fragmentation occurred.

Workflow of Transparent Fragmentation:

- * A large packet arrives at an ingress router (R) of a network (N).
- * R determines the packet is too large for N's MTU.
- * R fragments the packet into smaller pieces.
- * All fragments are routed within N to a specific exit router (RE).
- * RE collects all fragments.
- * RE reassembles the fragments into the original packet.
- * RE forwards the original packet to the next network.

Drawbacks:

- * **Overload on Exit Router:** All fragments must converge on a single exit router for reassembly. This can create a bottleneck, potentially overloading that router and slowing down packet forwarding, even if other routers are available for parallel processing.
- * **Increased Overhead:** If a packet traverses multiple networks, each requiring fragmentation, it could undergo multiple cycles of fragmentation and reassembly. This repeated processing adds significant overhead to the network.
- * **Fragment Tracking:** The exit router needs mechanisms to know when all fragments have arrived. This can be achieved using a "count field" in each fragment (indicating remaining fragments) or a "delimiter" (marking the last fragment).

3. Non-Transparent Fragmentation: End-to-End Responsibility

Non-transparent fragmentation is the approach adopted by the IP protocol. In this method, fragmentation is not transparent to subsequent networks; they are aware that fragmentation has occurred. When a router fragments a packet, the resulting

fragments are treated as independent IP packets and are forwarded individually to the next network. Crucially, there is no reassembly at any intermediate router.

The responsibility for reassembling the fragments lies solely with the final destination host. All fragments, potentially arriving out of order and via different paths, will eventually reach the IP layer of the destination computer. The destination host's IP layer then collects these fragments and reassembles them into the original packet.

Workflow of Non-Transparent Fragmentation:

- * A large packet is generated by a host in Network N1.
- * An intermediate router (e.g., between N1 and N2) fragments the packet if N2's MTU is smaller.
- * These fragments are forwarded independently to the next network.
- * If a fragment encounters another network with an even smaller MTU, it can be further fragmented.
- * All fragments (and sub-fragments) are routed independently towards the final destination.
- * The final destination host's IP layer receives all fragments.
- * The destination host reassembles the fragments into the original packet.

Advantages:

- * **Parallel Processing:** Fragments can be routed through multiple paths and exit routers, leading to better utilization of network resources and potentially higher throughput.
- * **Reduced Intermediate Overhead:** Intermediate routers only perform fragmentation (if needed) and forwarding, avoiding the computational burden of reassembly.

Drawbacks:

- * **Increased Header Overhead:** Each fragment is an independent IP packet and thus carries its own IP header (minimum 20 bytes). If an original large packet is broken into many fragments, the total number of header bytes transmitted across the network increases significantly. For example, an original 1020-byte packet (1000 data + 20 header) fragmented into four pieces might result in 1080 total bytes transmitted (1000 data + 4 * 20 headers).
- * **Destination Complexity:** The destination host must manage the reassembly process, including handling out-of-order arrival and potential loss of fragments.

4. IP Header Fields for Fragmentation Control

The IP header contains specific fields that are critical for managing fragmentation and reassembly in non-transparent fragmentation:

- **Identification (ID)** : This is a 16-bit field that uniquely identifies the original datagram. All fragments derived from the same original packet will share the same **ID** . This allows the destination host to group all related fragments together for reassembly. For instance, if an original packet has **ID** = 5 , all its fragments will also have **ID** = 5 .
- **Fragment Offset** : This 13-bit field indicates the position of the current fragment's data payload relative to the beginning of the original unfragmented data. The offset is specified in multiples of 8 bytes. This 8-byte multiple restriction is a design choice to conserve bits in the header while still allowing for large offsets. For example, if the first fragment carries 1000 bytes of data, the next fragment's offset would be $1000/8 = 125$. This field is crucial for the destination host to correctly order the fragments during reassembly.
- **Flags** : This 3-bit field contains two actively used flags:
 - **D (Do Not Fragment) Flag**: If this flag is set to 1, it instructs routers not to fragment the packet. If a router encounters a packet with the **D** flag set and its size exceeds the MTU of the next network, the router will drop the packet and typically send an ICMP error message back to the source. This is useful for applications that cannot tolerate fragmentation or for path MTU discovery.
 - **M (More Fragments) Flag**: If this flag is set to 1, it indicates that there are more fragments following the current one. If **M** is set to 0, it signifies that the current fragment is the last fragment of the original packet. This helps the destination host determine when it has received all parts of the original datagram.

5. Illustrative Example of IP Fragmentation

Consider an original IP packet with 1000 bytes of data and a standard 20-byte IP header, making a total size of 1020 bytes. Let its **ID** be 5, **Fragment Offset** be 0, and **M** flag be 0 (initially, as it's a single, unfragmented packet).

Scenario:

1. **Network 1 (MTU 620 bytes)**: The packet enters a network with an MTU of 620 bytes. Since $1020 > 620$, fragmentation is required.

* **Fragment 1**:

- * Data: 600 bytes (maximum data that fits with 20-byte header: $620 - 20 = 600$)
- * Header: 20 bytes
- * Total Size: 620 bytes

* ID : 5

* Fragment Offset : 0 (starts at the beginning)

* M Flag: 1 (more fragments are coming)

* **Fragment 2:**

* Data: 400 bytes (remaining data: $1000 - 600 = 400$)

* Header: 20 bytes

* Total Size: 420 bytes

* ID : 5

* Fragment Offset : $600 / 8 = 75$ (offset in 8-byte multiples)

* M Flag: 0 (this is the last fragment from the original packet)

1. **Network 2 (MTU 400 bytes):** Fragment 1 (620 bytes total) now enters a network with an MTU of 400 bytes.

2. **Fragment 1.1 (from original Fragment 1):**

3. Data: 376 bytes (maximum data that fits with 20-byte header and is a multiple of 8: $400 - 20 = 380$, nearest multiple of 8 is 376)

4. Header: 20 bytes

5. Total Size: 396 bytes

6. ID : 5

7. Fragment Offset : 0

8. M Flag: 1

9. **Fragment 1.2 (from original Fragment 1):**

10. Data: 224 bytes (remaining data from Fragment 1: $600 - 376 = 224$)

11. Header: 20 bytes

12. Total Size: 244 bytes

13. ID : 5

14. Fragment Offset : $376 / 8 = 47$

15. M Flag: 0 (this is the last part of original Fragment 1)

16. **Network 2 (MTU 400 bytes):** Fragment 2 (420 bytes total) also enters this network.

17. **Fragment 2.1 (from original Fragment 2):**

18. Data: 376 bytes

19. Header: 20 bytes

20. Total Size: 396 bytes

21. ID : 5
22. Fragment Offset : 75 (inherits offset from original Fragment 2)
23. M Flag: 1
24. **Fragment 2.2 (from original Fragment 2):**
25. Data: 24 bytes (remaining data from Fragment 2: $400 - 376 = 24$)
26. Header: 20 bytes
27. Total Size: 44 bytes
28. ID : 5
29. Fragment Offset : $(75 * 8 + 376) / 8 = 122$
30. M Flag: 0

In this example, an original 1020-byte packet results in four fragments. The total bytes transmitted become $396 + 244 + 396 + 44 = 1080$ bytes. This demonstrates the header overhead associated with non-transparent fragmentation.

6. IP and Ethernet MTU Considerations

The IP protocol allows for a maximum packet size of approximately 64 KB (65,535 bytes). However, IP often runs over Ethernet, which operates at the data link layer and has a maximum frame size (MTU) of 1500 bytes. This disparity means that if an IP packet exceeds 1500 bytes, it must be fragmented before being encapsulated into Ethernet frames.

To avoid this additional fragmentation at the Ethernet level, many IP implementations and network configurations impose a practical limit on the IP packet size, often setting the IP MTU to 1500 bytes. This ensures that most IP packets can pass through the Ethernet layer without further splitting. Since the majority of data packets generated are typically smaller than 1500 bytes, this limitation rarely impacts performance. For the rare cases of larger packets, they are fragmented at the IP layer itself into pieces of 1500 bytes or less, allowing them to seamlessly traverse Ethernet networks.

Key Takeaways

- [] IP fragmentation is necessary when a packet's size exceeds a network's Maximum Transfer Unit (MTU).
- [] The IP layer operates as a connectionless, unreliable datagram service, with reliability handled by higher layers.
- [] Transparent fragmentation involves fragmentation and reassembly within the same network, making the process invisible to subsequent networks.

- [] Non-transparent fragmentation, used by the IP protocol, sends fragments independently to the final destination for reassembly.
- [] The Identification , Fragment Offset , and Flags fields in the IP header are crucial for managing fragmentation and reassembly.
- [] The Fragment Offset is specified in multiples of 8 bytes, and the M flag indicates if more fragments are expected.
- [] Non-transparent fragmentation can lead to increased header overhead due to each fragment carrying its own IP header.
- [] Practical IP MTU is often limited to 1500 bytes to align with Ethernet's maximum frame size and avoid additional fragmentation at the data link layer.

Conclusion

IP packet fragmentation and reassembly are fundamental mechanisms that underscore the adaptive and resilient nature of the Internet Protocol. By allowing large data units to be broken down and reassembled, IP ensures that information can traverse a heterogeneous global network composed of links with vastly different capacities. While transparent fragmentation offers simplicity for downstream networks, the IP protocol's adoption of non-transparent fragmentation highlights a design choice that prioritizes network efficiency and parallel processing, even at the cost of increased header overhead.

Understanding these fragmentation strategies, particularly the role of specific IP header fields, is essential for anyone delving into network architecture and security. This foundational knowledge not only explains how data moves across the internet but also sets the stage for comprehending more advanced topics in IP addressing and routing, which will be explored in subsequent discussions. The ability of IP to manage varying MTUs is a testament to its robust design, enabling the seamless communication that defines our modern digital world.